

CSE563 — Concept Sheet, Block A

Unit I: Basic Mathematics in Computer Science

Unit II: Vectors and Matrices

Theory reference — every concept, no numericals

School of Computer Science & Engineering

Session 2025–26



What This Sheet Is

A **theory-only** reference for Units I and II, which are delivered together as one teaching block. Every concept, definition, law and statement that can be examined in written (non-numerical) form appears here, in the order it is taught. There are no worked calculations — those live in the *Numerical Guides*, and practice questions live in the *Worksheets*.

Use this sheet for revision, for written/descriptive answers, and as the definition list to be memorised.

Contents

Unit I — Basic Mathematics in Computer Science

1 Number Theory

Core Definitions

Divisibility — for integers $a \neq 0$ and b , $a \mid b$ means $b = ak$ for some integer k . Then a is a *divisor* of b and b a *multiple* of a .

Division algorithm — for any integer a and positive integer d there exist *unique* integers q, r with $a = dq + r$ and $0 \leq r < d$.

Prime — an integer $p > 1$ whose only positive divisors are 1 and p . Otherwise *composite*.

GCD — the largest integer dividing both a and b . **LCM**: the smallest positive integer that both divide.

Coprime — $\gcd(a, b) = 1$. The integers share no prime factor.

Congruence — $a \equiv b \pmod{m}$ iff $m \mid (a - b)$, equivalently a and b leave the same remainder on division by m .

Residue class — $[r] = \{x \in \mathbb{Z} : x \equiv r \pmod{m}\}$. There are exactly m such classes, and they partition $\mathbb{Z}$.

Theorems to State

Fundamental Theorem of Arithmetic. Every integer $n > 1$ factors into primes, uniquely up to order.

Euclid's theorem. There are infinitely many primes. *Proof sketch:* if $p_1, \dots, p_k$ were all of them, $N = p_1 \cdots p_k + 1$ is divisible by none, contradiction.

Bézout's identity. For integers a, b not both zero there exist integers x, y with $ax + by = \gcd(a, b)$.

Fermat's little theorem. If p is prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$.

Euler's theorem. If $\gcd(a, n) = 1$ then $a^{\phi(n)} \equiv 1 \pmod{n}$, where ϕ is Euler's totient function.

Key identity. $\gcd(a, b) \cdot \text{lcm}(a, b) = ab$.

Existence of a modular inverse. a^{-1} exists modulo m if and only if $\gcd(a, m) = 1$.

Why Computer Science Needs This

Hashing — an index is computed as $\text{key} \bmod m$; a prime m spreads keys that share common factors.

RSA cryptography — security rests on the difficulty of factoring $n = pq$; correctness rests on Euler's theorem.

Pseudo-random generators — linear congruential generators iterate $x_{n+1} = (ax_n + c) \bmod m$.

Check digits — ISBN, credit-card (Luhn) and Aadhaar validation are congruence conditions.

Complexity of Euclid — $O(\log \min(a, b))$ divisions; the worst case is consecutive Fibonacci numbers (Lamé's theorem).

2 Set Theory

Core Definitions

Set — an unordered collection of distinct objects. Described by *roster* or *set-builder* notation.

Subset — $A \subseteq B$ iff every element of A lies in B . *Proper* subset: $A \subseteq B$ and $A \neq B$.

Set equality — $A = B$ iff $A \subseteq B$ and $B \subseteq A$ — this double inclusion is how equality is *proved*.

Power set — $\mathcal{P}(A)$, the set of all subsets of A . $|\mathcal{P}(A)| = 2^{|A|}$.

Cartesian product — $A \times B = \{(a, b) : a \in A, b \in B\}$; ordered, so $(1, 2) \neq (2, 1)$.

Disjoint sets — $A \cap B = \emptyset$.

Partition — a family of non-empty, pairwise disjoint subsets whose union is the whole set.

The Laws of Set Algebra

For all sets A, B, C within a universal set U :

Law	Union form	Intersection form
Identity	$A \cup \emptyset = A$	$A \cap U = A$
Domination	$A \cup U = U$	$A \cap \emptyset = \emptyset$
Idempotent	$A \cup A = A$	$A \cap A = A$
Complement	$A \cup \overline{A} = U$	$A \cap \overline{A} = \emptyset$
Commutative	$A \cup B = B \cup A$	$A \cap B = B \cap A$
Associative	$(A \cup B) \cup C = A \cup (B \cup C)$	$(A \cap B) \cap C = A \cap (B \cap C)$
Distributive	$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
De Morgan	$\overline{A \cup B} = \overline{A} \cap \overline{B}$	$\overline{A \cap B} = \overline{A} \cup \overline{B}$
Absorption	$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$

Duality. Every law has a twin obtained by swapping $\cup \leftrightarrow \cap$ and $\emptyset \leftrightarrow U$. The *identical* table governs Boolean algebra and propositional logic — learn it once, use it three times.

Principle of Inclusion–Exclusion

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

Rationale: elements in an overlap are counted once for each set containing them, so the repeats must be subtracted; elements in all three are then subtracted once too often and must be added back.

Connection. Sets are the language of databases (SQL UNION, INTERSECT, EXCEPT), of type systems, and of probability (Unit III, where events *are* sets and the addition rule *is* inclusion–exclusion).

3 Boolean Algebra

Core Definitions

Boolean algebra — the set $\{0, 1\}$ with operations AND ($\cdot$), OR ($+$) and NOT ($-$), obeying the same laws as set algebra with $\cup \rightarrow +$, $\cap \rightarrow \cdot$, $\emptyset \rightarrow 0$, $U \rightarrow 1$.

Literal — a variable or its complement.

Minterm — a product containing every variable exactly once (e.g. $\overline{A}BC$). It is 1 for exactly one input combination.

Maxterm — a sum containing every variable exactly once (e.g. $A + \overline{B} + C$). It is 0 for exactly one combination.

Sum of Products (SOP) — an OR of product terms; the *canonical* SOP ORs the minterms where $F = 1$, written $\sum m(\dots)$.

Product of Sums (POS) — an AND of sum terms; the canonical POS ANDs the maxterms

where $F = 0$, written $\prod M(\dots)$.

Functionally complete set — a set of gates able to express every Boolean function. {AND, OR, NOT} is one; NAND alone is another, as is NOR alone.

Laws Used for Simplification

- **Complement:** $x + \bar{x} = 1$, $x \cdot \bar{x} = 0$, $\bar{\bar{x}} = x$.
- **Absorption:** $x + xy = x$, $x(x + y) = x$.
- **Adjacency (combining):** $xy + x\bar{y} = x$ — this is precisely what a K-map group performs visually.
- **Redundancy:** $x + \bar{x}y = x + y$.
- **Consensus:** $xy + \bar{x}z + yz = xy + \bar{x}z$ (the third term is implied and may be deleted).
- **De Morgan:** $\overline{x + y} = \bar{x}\bar{y}$, $\overline{xy} = \bar{x} + \bar{y}$.

Karnaugh Maps — the Principles

- Cells are arranged in **Gray-code** order (00,01,11,10) so that horizontally or vertically adjacent cells differ in exactly one variable.
- A group of 2^k adjacent 1s eliminates k variables — those that change within the group.
- Groups **may wrap** around the left/right edges, the top/bottom edges, and hence the four corners of a 4-variable map.
- Use the *fewest, largest* groups that cover every 1. Groups may overlap.
- A **prime implicant** is a group that cannot be enlarged; an **essential** prime implicant is one covering a 1 that no other group covers.
- **Don't-care** conditions ($\times$) may be treated as 1 when doing so enlarges a group, and as 0 otherwise.

Why minimise? Fewer literals means fewer gates, less silicon area, lower power and shorter propagation delay.

4 Logic

Propositional Logic

Proposition — a declarative sentence with a definite truth value, true or false but not both.

Connectives — $\neg$ (not), $\wedge$ (and), $\vee$ (or), $\rightarrow$ (implies), $\leftrightarrow$ (if and only if), $\oplus$ (exclusive or).

Implication — $p \rightarrow q$ is **false in exactly one case**: p true and q false. When p is false it is *vacuously true*.

Tautology — true under every assignment. **Contradiction**: false under every assignment.

Contingency: neither.

Logical equivalence — $P \equiv Q$ iff they have identical truth tables, iff $P \leftrightarrow Q$ is a tautology.

Converse, Inverse, Contrapositive

For $p \rightarrow q$:

Name	Form	Relationship to the original
Converse	$q \rightarrow p$	Not equivalent
Inverse	$\neg p \rightarrow \neg q$	Not equivalent (but equivalent to the converse)
Contrapositive	$\neg q \rightarrow \neg p$	Equivalent

Key rewriting law: $p \rightarrow q \equiv \neg p \vee q$. This converts every implication into something the

algebraic laws can manipulate.

Rules of Inference

Rule	Premises $\therefore$ Conclusion	Note
Modus ponens	$p, p \rightarrow q \therefore q$	the basic forward step
Modus tollens	$\neg q, p \rightarrow q \therefore \neg p$	reasoning backwards
Hypothetical syllogism	$p \rightarrow q, q \rightarrow r \therefore p \rightarrow r$	chaining implications
Disjunctive syllogism	$p \vee q, \neg p \therefore q$	elimination
Addition	$p \therefore p \vee q$	weakening
Simplification	$p \wedge q \therefore p$	extraction
Resolution	$p \vee q, \neg p \vee r \therefore q \vee r$	the engine of SAT solvers (Unit V)

Two fallacies to recognise and reject:

- *Affirming the consequent*: $q, p \rightarrow q \therefore p$. Invalid.
- *Denying the antecedent*: $\neg p, p \rightarrow q \therefore \neg q$. Invalid.

Predicate Logic and Quantifiers

Predicate — $P(x)$ — becomes a proposition once x takes a value from the *domain*.

Universal quantifier — $\forall x P(x)$ — “for all”. False as soon as one counterexample exists.

Existential quantifier — $\exists x P(x)$ — “there exists”. True as soon as one witness exists.

Negation rules — $\neg \forall x P(x) \equiv \exists x \neg P(x)$ and $\neg \exists x P(x) \equiv \forall x \neg P(x)$ — De Morgan for quantifiers.

Order matters — $\forall x \exists y (x < y)$ is true over $\mathbb{Z}$; $\exists y \forall x (x < y)$ is false. Swapping nested quantifiers changes the meaning.

5 Calculus for Computer Science

Concepts

Limit — $\lim_{x \rightarrow a} f(x) = L$ means $f(x)$ can be brought arbitrarily close to L by taking x close enough to a .

Continuity — f is continuous at a iff $\lim_{x \rightarrow a} f(x) = f(a)$.

Derivative — the instantaneous rate of change, $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$; geometrically the slope of the tangent.

Partial derivative — $\partial f / \partial x$ differentiates with respect to x holding the other variables constant.

Gradient — $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$ — a vector pointing in the direction of *steepest increase*.

Critical point — where $\nabla f = \mathbf{0}$. Classified by the second derivative: $f'' > 0$ minimum, $f'' < 0$ maximum, $f'' = 0$ inconclusive.

Integral — accumulation; the definite integral $\int_a^b f$ is the signed area under the curve, evaluated as $F(b) - F(a)$ where $F' = f$.

Rules and Standard Results

- $\frac{d}{dx} x^n = nx^{n-1}$; $\frac{d}{dx} e^x = e^x$; $\frac{d}{dx} \ln x = \frac{1}{x}$; $\frac{d}{dx} \sin x = \cos x$.
- **Product**: $(uv)' = u'v + uv'$. **Quotient**: $(u/v)' = (u'v - uv')/v^2$.
- **Chain rule**: $\frac{d}{dx} f(g(x)) = f'(g(x))g'(x)$ — the single most important rule, since *backpropagation is the chain rule applied layer by layer*.
- **Sigmoid derivative**: $\sigma(x) = \frac{1}{1+e^{-x}}$ has $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

- $\int x^n dx = \frac{x^{n+1}}{n+1} + C$ ($n \neq -1$); $\int \frac{1}{x} dx = \ln |x| + C$; $\int e^x dx = e^x + C$.

Gradient Descent — the Idea

Since ∇f points uphill, moving *against* it decreases the function:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k).$$

Learning rate α — too small $\rightarrow$ slow convergence; too large $\rightarrow$ oscillation or divergence. Diagnosed from the cost-versus-iteration curve.

Why it matters — every machine-learning model in CSE252/INT396 is trained by minimising a loss function this way. Unit III's least-squares regression is the case where the minimum can instead be found in closed form.

Connection to Unit V — $\sum_{k=1}^n \frac{1}{k} \approx \int_1^n \frac{dx}{x} = \ln n$, and $\sum_{k=1}^n \log k = \Theta(n \log n)$ — integrals are how summations in complexity analysis are bounded.

6 Mathematical Induction

The Principle

To prove $P(n)$ for all integers $n \geq n_0$:

1. **Basis step** — prove $P(n_0)$ directly.
2. **Inductive step** — assume $P(k)$ for an arbitrary $k \geq n_0$ (the *inductive hypothesis*) and deduce $P(k+1)$.

Conclude that $P(n)$ holds for all $n \geq n_0$. *Analogy*: the first domino falls, and each falling domino topples the next.

Strong induction assumes $P(n_0), \dots, P(k)$ all hold when proving $P(k+1)$. Use it when the step depends on cases further back than k — prime factorisation, Fibonacci-style recurrences, divide-and-conquer correctness. The number of base cases must match how far back the step reaches.

Where Induction Is Used in Computing

Loop invariants — a property true before the loop (basis), preserved by each iteration (step), and therefore true at exit. This is induction on the iteration count.

Recursive algorithm correctness — merge sort is correct on input of size n because it is correct on the two halves — strong induction on input size.

Recurrence solutions — the *substitution method* in Unit V guesses a closed form and verifies it by induction. It is the only one of the three methods that constitutes a proof.

Structural induction — proving properties of recursively defined structures (trees, lists, grammars, well-formed formulas).

Two Classic Errors

- **Omitting the basis.** The statement “ $n = n + 1$ ” has a perfectly valid inductive step and is false throughout.
- **A step that fails at the smallest k .** The notorious “all horses are the same colour” argument breaks at $k = 1 \rightarrow 2$, where the two overlapping subsets are in fact disjoint. Always confirm the step works at the *first* value of k .

7 Relations and Functions

Relations — Definitions and Properties

Binary relation — a subset $R \subseteq A \times B$. A relation *on* A is a subset of $A \times A$, representable as a digraph or a 0/1 matrix.

Property	Condition	Digraph / matrix picture
Reflexive	$\forall a : aRa$	a loop at every vertex; all-ones diagonal
Irreflexive	$\forall a : \neg aRa$	no loops; all-zeros diagonal
Symmetric	$aRb \Rightarrow bRa$	every arrow has a reverse; $M = M^T$
Antisymmetric	$aRb \wedge bRa \Rightarrow a = b$	no two-way arrow between distinct vertices
Transitive	$aRb \wedge bRc \Rightarrow aRc$	every two-step path has a shortcut

Caution: symmetric and antisymmetric are *not* opposites. Equality is both; $\{(1, 2), (2, 1), (1, 3)\}$ is neither.

Two Special Kinds of Relation

Equivalence relation — reflexive + symmetric + transitive. It **partitions** the set into disjoint *equivalence classes*, and conversely every partition induces an equivalence relation. Examples: equality, congruence mod m , “same connected component”. This is the mathematics behind SQL GROUP BY and the union-find data structure.

Partial order — reflexive + antisymmetric + transitive; the pair $(S, \preceq)$ is a *poset*. Examples: $(\mathbb{Z}, \leq)$, $(\mathcal{P}(S), \subseteq)$, $(\mathbb{Z}^+, |)$. Two elements are *comparable* if one precedes the other; if all pairs are comparable it is a **total order**.

Hasse diagram — a drawing of a poset with loops and transitively-implied edges removed, larger elements placed above smaller.

Bounds — *minimal* = nothing strictly below; *least* = below everything (unique if it exists). *glb* and *lub* of a pair; for divisibility these are gcd and lcm, making the divisor poset a *lattice*.

Transitive closure — $R^* = R \cup R^2 \cup \dots$, the smallest transitive relation containing R . In a graph, $(a, b) \in R^*$ iff a path exists from a to b ; computed by Warshall’s algorithm in $O(n^3)$.

Functions

Function — $f : A \rightarrow B$ assigns to *every* $a \in A$ *exactly one* $b \in B$. A is the domain, B the codomain, $f(A)$ the range.

Injective (one-to-one) — $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$; distinct inputs give distinct outputs.

Surjective (onto) — range = codomain; every element of B is attained.

Bijective — both; equivalently an inverse f^{-1} exists. Between finite sets a bijection exists iff $|A| = |B|$.

Composition — $(g \circ f)(x) = g(f(x))$ — inner function first. Associative but *not* commutative; $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$.

Floor and ceiling — $\lfloor x \rfloor$ = greatest integer $\leq x$; $\lceil x \rceil$ = least integer $\geq x$. Note $\lfloor -3.2 \rfloor = -4$.

Pigeonhole principle — if n items go into $m < n$ boxes, some box holds at least two. This is why a hash function on more keys than slots *must* collide.

Unit II — Vectors and Matrices

8 Vectors

Definitions

Vector — an ordered n -tuple $\mathbf{v} \in \mathbb{R}^n$; geometrically an arrow with magnitude and direction, algebraically a point in n -space.

Dot (inner) product — $\mathbf{u} \cdot \mathbf{v} = \sum u_i v_i = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$. A *scalar*.

Norm — $\|\mathbf{v}\|_2 = \sqrt{\sum v_i^2}$ (Euclidean). Also $\|\mathbf{v}\|_1 = \sum |v_i|$ (Manhattan) and $\|\mathbf{v}\|_\infty = \max |v_i|$.

Unit vector — $\hat{\mathbf{v}} = \mathbf{v} / \|\mathbf{v}\|$, of length 1.

Orthogonal — $\mathbf{u} \cdot \mathbf{v} = 0$; the vectors are perpendicular.

Projection — $\text{proj}_{\mathbf{u}} \mathbf{v} = \frac{\mathbf{v} \cdot \mathbf{u}}{\|\mathbf{u}\|^2} \mathbf{u}$ — the shadow of $\mathbf{v}$ along $\mathbf{u}$.

Cross product (3-D only) — $\mathbf{u} \times \mathbf{v}$ is perpendicular to both, with magnitude $\|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$ = the area of the parallelogram. Used for surface normals in graphics.

Spans, Independence and Bases

Linear combination — $c_1 \mathbf{v}_1 + \dots + c_k \mathbf{v}_k$. The set of all of them is the **span**.

Linear independence — $c_1 \mathbf{v}_1 + \dots + c_k \mathbf{v}_k = \mathbf{0}$ only when every $c_i = 0$ — no vector is expressible in terms of the others.

Basis — a linearly independent set that spans the space. $\mathbb{R}^n$ needs exactly n basis vectors; every vector then has a *unique* representation.

Orthonormal basis — basis vectors that are mutually orthogonal and of unit length. PCA constructs exactly such a basis, which is why its projected coordinates are uncorrelated.

Dimension — the number of vectors in any basis.

What the Dot Product Tells You — and Why It Matters

- Sign of $\mathbf{u} \cdot \mathbf{v}$: positive $\Rightarrow$ angle under 90° ; zero $\Rightarrow$ perpendicular; negative $\Rightarrow$ over 90° .
- $\mathbf{u} \cdot \hat{\mathbf{v}}$ is the length of the shadow of $\mathbf{u}$ on the direction $\hat{\mathbf{v}}$. **This single fact is PCA** — projecting data onto a principal direction is a dot product.
- **Cosine similarity** = $\cos \theta$ compares documents or embeddings independently of their magnitude, which is why it is the default similarity in text retrieval and recommender systems.

9 Matrices

Definitions and Special Types

Matrix — a rectangular $m \times n$ array $A = [a_{ij}]$.

Identity I — ones on the diagonal, zeros elsewhere; $AI = IA = A$.

Symmetric — $A = A^T$. **Crucially**, a real symmetric matrix has real eigenvalues and orthogonal eigenvectors — the spectral theorem.

Orthogonal matrix — $A^T A = I$, so $A^{-1} = A^T$; its columns form an orthonormal basis. Rotation matrices are orthogonal.

Triangular — all entries above (lower) or below (upper) the diagonal are zero. Its eigenvalues are simply its diagonal entries.

Singular — $\det A = 0$; no inverse exists.

Rank — the number of linearly independent rows (equivalently, columns). A square matrix is invertible iff it has full rank.

Trace — $\text{tr } A = \sum a_{ii}$, which equals the sum of the eigenvalues.

Operations and Their Algebra

- **Product:** $(AB)_{ij}$ = row i of A dotted with column j of B ; defined only when the inner dimensions match. Cost $O(mnp)$.
- **Not commutative:** $AB \neq BA$ in general — but associative, $(AB)C = A(BC)$, and distributive.
- **Transpose:** $(AB)^T = B^T A^T$ — the order reverses.
- **Inverse:** exists iff $\det A \neq 0$; $(AB)^{-1} = B^{-1} A^{-1}$ — again the order reverses.
- **Determinant:** $\det(AB) = \det A \det B$; $\det A^T = \det A$. Geometrically $|\det A|$ is the volume scaling factor of the transformation, and a negative determinant signals orientation reversal (a reflection).

Linear Systems

For $A\mathbf{x} = \mathbf{b}$:

- **Unique solution** iff A is square and non-singular; then $\mathbf{x} = A^{-1}\mathbf{b}$ in theory, but Gaussian elimination ($O(n^3)$) in practice — never form the inverse numerically.
- **Consistency:** a solution exists iff $\text{rank}(A) = \text{rank}[A \mid \mathbf{b}]$; it is unique iff that rank equals the number of unknowns.
- **Cramer's rule:** $x_i = \det(A_i) / \det(A)$, practical only for very small systems.
- **Overdetermined systems** (more equations than unknowns) are solved in the least-squares sense by $\mathbf{x} = (A^T A)^{-1} A^T \mathbf{b}$ — this is exactly multiple regression in Unit III.

Eigenvalues and Eigenvectors

Definition — $\mathbf{v} \neq \mathbf{0}$ is an eigenvector with eigenvalue λ if $A\mathbf{v} = \lambda\mathbf{v}$ — the matrix merely *stretches* $\mathbf{v}$ without rotating it.

Characteristic equation — $\det(A - \lambda I) = 0$. For 2×2 it reduces to $\lambda^2 - (\text{tr } A)\lambda + \det A = 0$.

Identities — $\sum \lambda_i = \text{tr } A$ and $\prod \lambda_i = \det A$ — the standard arithmetic checks.

Diagonalisation — if A has n independent eigenvectors then $A = PDP^{-1}$ with D diagonal, whence $A^k = PD^k P^{-1}$.

Applications — PageRank is the dominant eigenvector of the web link matrix; PCA is the eigen-decomposition of a covariance matrix; vibration modes and stability analysis are eigenproblems.

10 Principal Component Analysis

The Problem and the Idea

Data with p features live in $\mathbb{R}^p$, but most of their *variance* usually lies along a few directions. PCA finds an orthonormal set of directions — the **principal components** — ordered by the variance they capture, and projects the data onto the first k .

Procedure (conceptually). Centre the data $\rightarrow$ form the covariance matrix $\rightarrow$ eigen-decompose it $\rightarrow$ order the eigenvalues $\rightarrow$ keep the top k eigenvectors $\rightarrow$ project.

Optimality. The first principal component is the unit direction maximising $\text{Var}(\tilde{X}\mathbf{w})$, and the maximum attained is λ_1 . Equivalently, it is the direction minimising the total squared reconstruction error.

Properties and Interpretation

Variance explained — component i accounts for $\lambda_i / \sum_j \lambda_j$ of the total variance. A *scree plot* or a cumulative threshold (e.g. 95%) chooses k .

Orthogonality — because the covariance matrix is symmetric, the components are guaranteed orthogonal and the variances add.

Zero eigenvalue — indicates an exactly redundant direction — the data lie in a lower-dimensional subspace.

Lossy reconstruction — $\hat{X} = ZW_k^T + \bar{\mathbf{x}}$; the discarded components are precisely the reconstruction error.

Common Pitfalls

- **Not centring** — the “first component” then points at the mean rather than along the variance.
- **Not scaling features with different units** — a feature measured in metres will dominate one in kilometres purely by magnitude.
- **Assuming interpretability** — components are linear mixtures of the original features and often have no physical meaning.
- **Forgetting PCA is unsupervised** — it never sees class labels, so a high-variance direction need not be discriminative. That is LDA’s job.
- **Sign ambiguity** — $\mathbf{w}$ and $-\mathbf{w}$ are equally valid eigenvectors.

11 Linear Discriminant Analysis

Fisher’s Criterion

LDA is **supervised**: given labelled samples, it finds the direction $\mathbf{w}$ onto which projection best *separates the classes*, maximising

$$J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}} = \frac{(\text{separation between projected class means})^2}{(\text{scatter within the projected classes})}.$$

Within-class scatter — $S_W = \sum_i S_i$, where $S_i = \sum_{\mathbf{x} \in C_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T$.

Between-class scatter — $S_B = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T$ for two classes.

Solution — $\mathbf{w}^* \propto S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$. For C classes, LDA yields at most $C - 1$ directions, the eigenvectors of $S_W^{-1} S_B$.

Key intuition — dividing by S_W is what distinguishes LDA from simply joining the class means: directions in which the classes are internally noisy get *down-weighted*.

PCA versus LDA — the Comparison Table

Aspect	PCA	LDA
Supervision	Unsupervised (ignores labels)	Supervised (requires labels)
Objective	Maximise projected variance	Maximise class separation relative to within-class spread
Decomposes	Covariance matrix S	$S_W^{-1} S_B$
Max components	$\min(n - 1, p)$	$C - 1$
Typical use	Compression, visualisation, denoising	Feature extraction before classification; Fisherfaces
Fails when	Discriminative direction has low variance	S_W is singular (too few samples per class)

12 Image Transformation and Interpolation

Images as Matrices

A grayscale image is a matrix $I \in \{0, \dots, 255\}^{H \times W}$; a colour image stacks three such matrices (R, G, B). The row index increases *downward*.

Point operation — changes intensities in place, e.g. brightness/contrast $I' = \alpha I + \beta$ clipped to $[0, 255]$.

Geometric transformation — moves pixels to new positions, $(x, y) \mapsto (x', y')$.

Homogeneous Coordinates — Why They Exist

A plain 2×2 matrix represents a *linear* map, which always fixes the origin and therefore cannot express translation. Appending a third coordinate 1 and using 3×3 matrices makes translation an ordinary matrix multiplication, so that **every** affine transformation composes by multiplying matrices.

Affine transformation — preserves straight lines and parallelism. General form $\begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix}$.

Composition order — to apply T_1 then T_2 , form $M = T_2 T_1$ — the **right-most matrix acts first**. Order matters, since matrix multiplication is not commutative.

Rotation about a point P — $M = T(P) R(\theta) T(-P)$ — translate the centre to the origin, rotate, translate back.

Rotation matrices — orthogonal, with $R^{-1} = R^T = R(-\theta)$ and $\det R = 1$, so lengths and areas are preserved. A reflection has $\det = -1$; a shear has $\det = 1$ (area preserved, shape slanted).

Homography — a 3×3 projective transform whose last row is not $(0, 0, 1)$; it handles perspective, used in panorama stitching.

Forward versus Inverse Mapping

Forward mapping — sends each source pixel to $M(x, y)$. Results land on non-integer positions, leaving some destination pixels unwritten (*holes*) and others written several times.

Inverse mapping — for each *destination* pixel, compute $M^{-1}(x', y')$ to find where it came from, then interpolate in the source. No holes — and this is what every real library does.

Consequence: because the computed source location is fractional, *interpolation is unavoidable* for any rotation, scaling or warp that is not a whole-pixel translation.

Interpolation Methods

Method	How it works	Character
Nearest neighbour	Copy the closest pixel	Fastest; blocky and jagged; <i>preserves exact original values</i> , so it is the correct choice for label/segmentation maps
Bilinear	Weighted average of the surrounding 2×2 , weights proportional to proximity	Smooth, slightly blurring; the real-time graphics default (and implemented in GPU hardware)
Bicubic	Fits cubic polynomials through the 4×4 neighbourhood	Sharpest, fewest artefacts, most expensive; the photo-editing default

Bilinear weights. With a and b the fractional offsets, the four weights are $(1 - a)(1 - b)$, $a(1 - b)$, $(1 - a)b$ and ab ; *they always sum to 1*.

Limitation to state in any answer. Interpolation cannot create information. Enlarging a low-resolution image yields a smooth result, not a detailed one — no method recovers detail that was never captured. Repeated resampling also accumulates blur, so transformations should

be *composed into one matrix* and the image resampled once.

💡 **Exam Tip & Key takeaway: Block A — The Thread That Connects Both Units**

Unit I supplies the language and the proof techniques; Unit II supplies the computational machinery.

- **De Morgan** appears in three costumes — sets, Boolean algebra, logic — and again for quantifiers. Learning the law once covers all four.
- **Equivalence relations** (Unit I) are what make congruence classes, and later the partitions that clustering produces.
- **The gradient** (Unit I calculus) is the tool that trains every model; **eigenvectors** (Unit II) are the tool that reshapes the data before training.
- **Symmetric matrices** guarantee orthogonal eigenvectors — the fact that makes PCA well defined.
- **Matrix composition order** in image transformation is the same non-commutativity first met in function composition ($g \circ f \neq f \circ g$).